

Tutorz Release & Deployment Guide

This guide outlines the standard operating procedure (SOP) for safely building, versioning, and releasing the Tutorz platform. Following these steps ensures your users always receive the latest code and your backend remains secure.

Phase 1: Pre-Release Version Bump

Phase 2: Building the Frontend (React + Vite PWA)

Phase 3: Building the Backend (.NET API)

Phase 4: Database Safety & Migrations

Phase 5: Security & Maintenance Best Practices

Phase 1: Pre-Release Version Bump

Before running any build commands, you must increment the version numbers so the system knows an update occurred.

1. Frontend Version

1. Open `d:\Projects\Tutorz\tutorz-frontend\src\config\version.js`
2. **Update the APP_VERSION string.**

Example: change '0.1.0-beta' to '0.2.0-beta'.

Segment	Type	When to Increment	Example
MAJOR	Breaking	Significant rewrite, new architecture, or massive branding shift.	1.0.0 , 2.0.0
MINOR	Feature	New modules/pages, database schema changes, or significant CSS updates.	0.5.0 , 0.6.0
PATCH	Hotfix	Small bug fixes, text corrections, or minor styling tweaks.	0.5.1 , 0.5.2

2. Backend Version

1. Open d:\Projects\Tutorz\tutorz-backend\Tutorz.Api\appsettings.json (and appsettings.Staging.json / appsettings.Development.json if necessary).
2. **Update the "AppVersion"** value to match the frontend.

Phase 2: Building the Frontend (React + Vite PWA)

The frontend must be compiled into static HTML, CSS, and JS files. During this process, the Service Worker (sw.js) is automatically regenerated with new unique file hashes.

1. Open a terminal and navigate to the frontend directory:
`cd D:\Projects\Tutorz\tutorz-frontend`
2. Install any new dependencies (if you added new packages):
`npm install`
3. Run the production build command:
`npm run build`
4. **Result:** A new folder called dist will be created inside the tutorz-frontend folder.

IMPORTANT

The contents of this dist folder are what you upload to your Azure App Service (or static web host). You do not upload the source code.

Phase 3: Building the Backend (.NET API)

The backend needs to be compiled and published into a secure, executable package that doesn't include your raw source code.

1. Open a terminal and navigate to the backend directory:
`cd D:\Projects\Tutorz\tutorz-backend`
2. Run the secure publish command for Azure:
`dotnet publish Tutorz.Api/Tutorz.Api.csproj -c Release -o ./publish /p:UseAppHost=false`

Explanation of flags:

- **-c Release**: Optimizes the code for speed and removes developer debugging data.
 - **-o ./publish**: Outputs the final files into a folder named publish.
 - **/p:UseAppHost=false**: Best practice for Azure App Services (prevents creating unnecessary OS-specific executables).
3. **Result**: A folder named publish is created. This folder contains all the .dll files, appsettings.json, and static web assets .

IMPORTANT

The contents of this publish folder are what you upload to your Azure App Service for the API.

Phase 4: Database Safety & Migrations

If your release includes changes to the C# Domain Entities you must prepare the database.

1. Generate a migration (run this *before* publishing if you made changes) in **Package Manager Console**:
Add-Migration [Name]
Creates a new migration script based on changes in your DbContext and entities.
Example: **Add-Migration InitialCreate**
2. Update the database (can be done locally targeting Azure or auto-run on startup depending on your Azure configuration) in **Package Manager Console**:
Update-Database
Applies any pending migrations to your SQL Server database.
Example: **Update-Database -TargetMigration "MigrationName"** (to roll back or move to a specific version).

Phase 5: Security & Maintenance Best Practices

To keep the Tutorz system secure in production, strictly observe these rules

CAUTION

Never commit Live App Setting hardcoded secrets Remove all passwords, JWT Keys, Database Connection Strings, and Email passwords from appsettings.json before committing to GitHub.

Secure Configuration via Azure

Instead of putting real passwords in appsettings.json during the build, use Azure Environment Variables:

1. Go to your Azure API App Service -> Settings -> Environment variables.
2. Add your secrets there (e.g., Name: Jwt__Key, Value: your-real-production-key-here).
3. Azure will automatically inject these into the running application, overriding anything in the appsettings.json file. This means if someone steals your build package, they still don't get your database password.

Emergency Forced Logout (Layer 3)

If a critical security flaw is discovered or you change the database schema massively and old user tokens are causing backend crashes

1. Open Azure Data Studio or SQL Server Management Studio.
2. Connect to the Tutorz Azure SQL Database.
3. Run this query to instantly **log out** all active users **across Sri Lanka**

```
UPDATE AppSettings
SET Value = CONVERT(varchar, GETUTCDATE(), 127), UpdatedAt = GETUTCDATE()
WHERE [Key] = 'MinTokenDate'
```